TUTORIAL DE GIT · CAPÍTULO 1

Inicio: sobre el control de versiones

Qué es Git, cómo funciona internamente y cómo dejarlo listo para trabajar

1. ¿Qué es el control de versiones?

Un **sistema de control de versiones (VCS)** registra los cambios hechos sobre un conjunto de archivos a lo largo del tiempo, de modo que se pueda recuperar una versión específica más adelante. Permite revertir archivos o proyectos completos a un estado anterior, comparar cambios, saber quién modificó qué y cuándo, y recuperarse de errores sin perder el trabajo.

Tipos de sistemas de control de versiones

Tipo	Cómo funciona	Ejemplos
Local	Una base de datos simple en el mismo computador guarda los cambios de los archivos. No sirve para colaborar.	RCS
Centralizado (CVCS)	Un único servidor guarda todas las versiones y los clientes descargan los archivos desde allí. Si el servidor falla, se pierde la historia.	CVS, Subversion, Perforce
Distribuido (DVCS)	Cada cliente replica el repositorio completo, con toda su historia. Si un servidor falla, cualquier copia sirve para restaurarlo.	Git, Mercurial, Bazaar, Darcs

2. Breve historia de Git

El núcleo de Linux usó durante años una herramienta propietaria llamada BitKeeper. En 2005 la relación con la empresa que la desarrollaba se rompió y se retiró el uso gratuito, por lo que **Linus Torvalds** y la comunidad crearon su propio sistema: Git. Sus metas fueron: velocidad, diseño sencillo, buen soporte para el desarrollo no lineal (miles de ramas en paralelo), ser completamente distribuido y poder manejar proyectos muy grandes.

3. Fundamentos de Git

- **Instantáneas, no diferencias.** Git ve sus datos como una serie de instantáneas (*snapshots*) del sistema de archivos. En cada commit guarda una "foto" del estado de todos los archivos y, si un archivo no cambió, solo guarda un enlace a la versión anterior.
- **Casi todas las operaciones son locales.** Como se tiene toda la historia en el disco, consultar el historial o comparar versiones es instantáneo y se puede trabajar sin conexión.
- **Integridad.** Todo se verifica con una suma de comprobación (*checksum*) SHA-1 antes de almacenarse; es una cadena de 40 caracteres hexadecimales. Es imposible cambiar un archivo sin que Git lo detecte.
- **Generalmente solo añade datos.** Casi todas las acciones agregan información a la base de datos, por lo que es difícil perder trabajo que ya fue confirmado.

Los tres estados de un archivo

Estado	Significado
Modificado (*modified*)	Se cambió el archivo, pero todavía no se confirmó en la base de datos.
Preparado (*staged*)	Se marcó el archivo modificado para que vaya en la próxima confirmación.
Confirmado (*committed*)	Los datos están almacenados de forma segura en la base de datos local.

Estos estados corresponden a tres secciones de un proyecto Git: el **directorio de trabajo** (copia de una versión del proyecto), el **área de preparación** (*staging area*, un archivo que indica qué irá en el próximo commit) y el **directorio de Git** (`.git`, donde se guardan los metadatos y la base de objetos).

```
Directorio de trabajo --(git add)--> Área de preparación --(git commit)-->
Directorio .git
```

4. La línea de comandos

Existen interfaces gráficas para Git, pero la línea de comandos es la única que ofrece **todos** los comandos; quien sabe usarla puede manejar cualquier interfaz gráfica, pero no al revés. Por eso esta actividad se desarrolla en la terminal (Git Bash en Windows).

5. Instalación de Git

Sistema	Cómo instalarlo
Windows	Descargar el instalador desde https://git-scm.com/download/win (incluye Git Bash).
macOS	Ejecutar `git --version` (ofrece instalar las herramientas de línea de comandos) o usar `brew install git`.
Linux (Debian/Ubuntu)	`sudo apt install git`
Linux (Fedora)	`sudo dnf install git`

Para comprobar la instalación:

```
git --version
```

6. Configuración inicial

La herramienta `git config` permite leer y definir variables de configuración. Existen tres niveles, y cada uno sobrescribe al anterior:

- `--system`: aplica a todos los usuarios del equipo (archivo `/etc/gitconfig`).
- `--global`: aplica al usuario actual (archivo `~/.gitconfig`).
- `--local`: aplica solo al repositorio actual (archivo `.git/config`).

Identidad

Lo primero es definir el nombre y el correo, porque Git los incluye en cada commit:

```
git config --global user.name "Nombre Apellido"
git config --global user.email "correo@ejemplo.com"
```

Editor, rama inicial y comprobación

```
git config --global core.editor "code --wait"   # editor para los mensajes de
commit
git config --global init.defaultBranch main      # nombre de la rama inicial
git config --list                                 # ver toda la configuración
git config user.name                              # ver un valor concreto
```

7. Cómo obtener ayuda

```
git help <verbo>
git <verbo> --help
git <verbo> -h        # resumen corto de opciones

# Ejemplo
git help config
```

> *Resumen: Git es un sistema distribuido que guarda instantáneas, trabaja casi todo en local y garantiza la integridad con SHA-1. Todo archivo pasa por tres estados (modificado, preparado y confirmado), y antes de usarlo hay que configurar la identidad con `git config`.*

Fuente: Adaptado de Chacon, S. y Straub, B. (2014). Pro Git (2.ª ed.). Apress. https://git-scm.com/book/es/v2 (licencia CC BY-NC-SA 3.0).